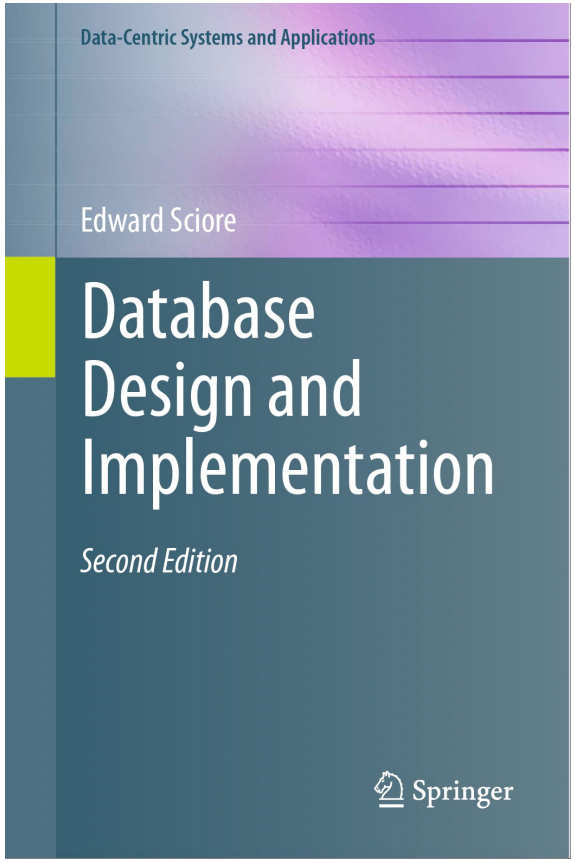
SimpleDB, parte 4

Luigi Bellomarini
luigi.bellomarini@uniroma3.it



Esercizio pratico 3.1

- Consideriamo il seguente setting (una variante dell'esercizio d'esame Prova parziale del 30/03/2015, domanda 1).
- R è una relazione definita su due campi A (di tipo intero) e B (di tipo stringa), di lunghezza:
  - a = 4 byte; b = 12 caratteri
- R è memorizzata sulla struttura fisica sequenziale di SimpleDB
- Utilizziamo blocchi di 2000 byte e un buffer di 500 blocchi
- Utilizziamo la strategia di rimpiazzo naive.

- Implementare le seguenti operazioni interagendo direttamente con le classi del package **record**:
  - **Inserimento** di 100.000 ennuple con valori progressivi per A a partire da 0 e stringhe casuali per B.
    - Visualizzare il numero di blocchi scritti per creare la relazione
  - **Cancellare** le ennuple per le quali 0<=A<20.000
    - Visualizzare il numero di blocchi cancellati
  - **Inserimento** di 10.000 ennuple con valori casuali di A compresi tra 0 e 100 a partire dal primo blocco
    - Visualizzare il numero di blocchi scritti per effettuare l'inserimento
  - Generare un insieme S di 500 numeri casuali compresi tra 0 e 1000 e **contare il numero delle ennuple** per le quali il valore di A appartiene a S
    - Visualizzare il numero di blocchi letti per effettuare il conteggio.

Esercizio pratico 3.1

- Abbiamo B=2000 (dimensione del blocco) e BUF=500 pagine di buffer
- Dimensione di un record RL= 4 byte (used flag) + 4 byte (A intero) + 12 byte (B stringa) + 4 byte (intero per lunghezza stringa) = 24 byte
- Blocchi scritti per **l'inserimento dei primi** R=100.000 record
  - Consideriamo che in SimpleDB la memorizzazione è di tipo unspanned, quindi F = ⌊B/RL⌋ = ⌊2000 / 24⌋ = 83 record
  - Da cui il numero di blocchi necessari per la memorizzazione è ⌈R/F⌉ = ⌈100000 / 83⌉ = 1205 blocchi
  - Quanti ne verranno scritti? Abbiamo 500 pagine di buffer, ma stiamo utilizzando la strategia naive
  - Veranno quindi scritti 1205-1 = 1204 blocchi: ad ogni unpin si scarica una pagina, quindi tutte tranne l'ultima.
- Blocchi scritti per la **cancellazione delle ennuple per cui 0<=A<=20.000**, cioè di R = 20000 record
  - I record sono memorizzati in ⌈R/F⌉ = ⌈20000 / 83⌉ = 241 blocchi
  - Vengono scritti 242 blocchi, poiché occorre salvare (flush) anche l'ultima pagina utilizzata per l'inserimento precedente
  - In questo caso l'ultimo blocco cancellato viene scritto perché occorre allocare pagine per terminare la scansione
- Blocchi scritti per l'**inserimento di 10.000 ennuple a partire dal primo blocco**
  - I record sono memorizzati in ⌈R/F⌉ = ⌈10000 / 83⌉ = 121 blocchi
  - Per la presenza del buffer ne verranno scritti 121-1 = 120 blocchi
- Blocchi letti per la **cancellazione** di ennuple data una condizione
  - Occorre leggere tutti i blocchi, 1205
  - Non si hanno benefici dal buffer (naive).

SimpleDB, componenti "read-only"

JDBC: riceve le richieste dal client e passa SQL a Planner

Planner: chiama Parser e determina piano e lo passa al Query

Parser: analisi sintattica

Query: riceve il piano e chiama il Record per ogni tabella

Metadata: gestisce gli schemi delle tabelle

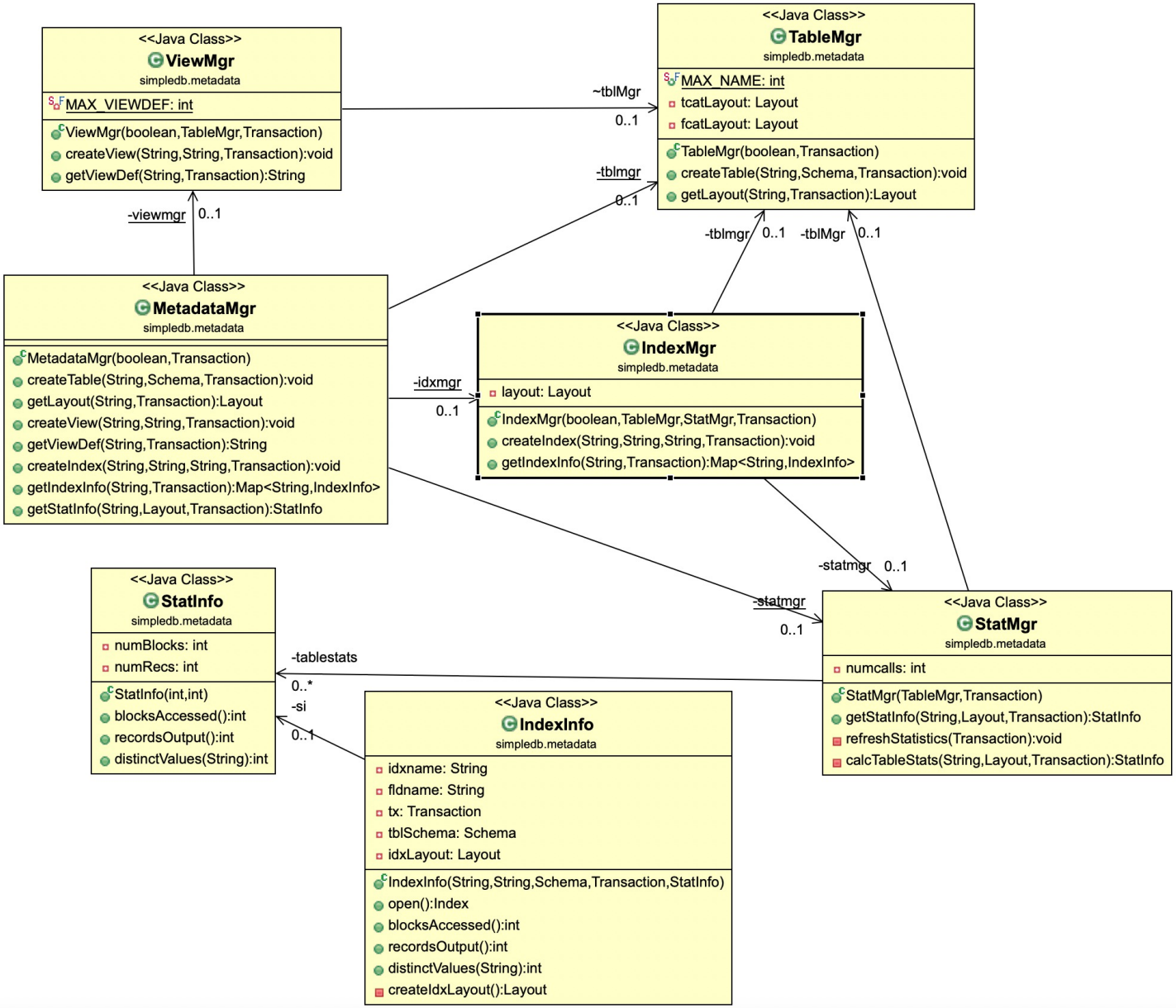
Record: gestisce i blocchi per i record delle tabelle

Transaction: gestisce la concorrenza e l'affidabilità

Buffer: mantiene in memoria le pagine per limitare gli accessi

Log: tiene traccia delle operazioni, per l'affidabilità

File: legge e scrive le pagine su disco


<<Java Class>>
ViewMgr
simpledb.metadata
MAX_VIEWDEF: int
ViewMgr(boolean,TableMgr,Transaction)
createView(String,String,Transaction):void
getViewDef(String,Transaction):String
~tblMgr
0..1
<<Java Class>>
TableMgr
simpledb.metadata
MAX_NAME: int
tcatLayout: Layout
fcatLayout: Layout
TableMgr(boolean,Transaction)
createTable(String,Schema,Transaction):void
getLayout(String,Transaction):Layout
-tblmgr
0..1
-viewmgr
0..1
-tblmgr
0..1
-tblMgr
0..1
<<Java Class>>
MetadataMgr
simpledb.metadata
MetadataMgr(boolean,Transaction)
createTable(String,Schema,Transaction):void
getLayout(String,Transaction):Layout
createView(String,String,Transaction):void
getViewDef(String,Transaction):String
createIndex(String,String,String,Transaction):void
getIndexInfo(String,Transaction):Map<String,IndexInfo>
getStatInfo(String,Layout,Transaction):StatInfo
-idxmgr
0..1
<<Java Class>>
IndexMgr
simpledb.metadata
layout: Layout
IndexMgr(boolean,TableMgr,StatMgr,Transaction)
createIndex(String,String,String,Transaction):void
getIndexInfo(String,Transaction):Map<String,IndexInfo>
-statmgr
0..1
-statmgr
0..1
<<Java Class>>
StatInfo
simpledb.metadata
numBlocks: int
numRecs: int
StatInfo(int,int)
blocksAccessed():int
recordsOutput():int
distinctValues(String):int
-tablestats
0..*
-si
0..1
<<Java Class>>
StatMgr
simpledb.metadata
numcalls: int
StatMgr(TableMgr,Transaction)
getStatInfo(String,Layout,Transaction):StatInfo
refreshStatistics(Transaction):void
calcTableStats(String,Layout,Transaction):StatInfo
<<Java Class>>
IndexInfo
simpledb.metadata
idxname: String
fldname: String
tx: Transaction
tblSchema: Schema
idxLayout: Layout
IndexInfo(String,String,Schema,Transaction,StatInfo)
open():Index
blocksAccessed():int
recordsOutput():int
distinctValues(String):int
createIdxLayout():Layout

Il catalogo

- *A database system stores its metadata in catalog tables*

Table: tblcat(TblName, RecLength)

Field: fdlcat(TblName, FldName, Type, Length, Offset)

View: viewcat(ViewName, ViewDef)

Index: idxcat(Indexname, Tablename, Fieldname)

Esempio di un catalogo

tblcat	TblName	RecLength
	student	26
	dept	16
	course	32
	section	28
	enroll	18

fldcat	TblName	FldName	Type	Length	Offset
	student	sid	4	0	0
	student	sname	12	10	4
	student	majorid	4	0	18
	student	gradyear	4	0	22
	dept	did	4	0	0
	dept	dname	12	8	4
	course	cid	4	0	0
	course	title	12	20	4
	course	deptid	4	0	28
	section	sectid	4	0	0
	section	courseid	4	0	4
	section	prof	12	8	8
	section	year	4	0	20
	enroll	eid	4	0	0
	enroll	studentid	4	0	4
	enroll	sectionid	4	0	8
	enroll	grade	12	2	12

Statistical metadata

T	B(T)	R(T)	V(T,F)
STUDENT	4,500	45,000	45,000 for F=SId 44,960 for F=SName 50 for F=GradYear 40 for F=MajorId
DEPT	2	40	40 for F=DId, DName
COURSE	25	500	500 for F=CId, Title 40 for F=DeptId
SECTION	2,500	25,000	25,000 for F=SectId 500 for F=CourseId 250 for F=Prof 50 for F=YearOffered
ENROLL	50,000	1,500,000	1,500,000 for F=EId 25,000 for F=SectionId 45,000 for F=StudentId 14 for F=Grade